

## **Student Grade Calculator using Python**

### **Beginner Python Project Documentation**

---

#### **Cover Page**

**Project Title:** Student Grade Calculator

**Subject:** Python Programming

**Student Name:** \_\_\_\_\_

**Roll Number:** \_\_\_\_\_

**Course:** \_\_\_\_\_

**College:** \_\_\_\_\_

**Submission Date:** \_\_\_\_\_

**Guide/Trainer:** \_\_\_\_\_

---

#### **Table of Contents**

1. Introduction
2. Project Overview
3. Objectives
4. Problem Statement
5. Theory Concepts Used
6. Software Requirements
7. Hardware Requirements
8. Project Structure
9. Algorithm
10. Flow of Program
11. Python Source Code
12. Explanation of Code
13. Sample Input & Output
14. Test Cases

- 15. Screenshots
- 16. Learning Outcomes
- 17. Advantages
- 18. Limitations
- 19. Future Enhancements
- 20. Conclusion

---

## 1. Introduction

Python is one of the easiest programming languages for beginners because of its simple syntax and readability. This project demonstrates the use of decision-making statements, loops, functions, and input validation by creating a Student Grade Calculator. The program accepts a student's name and marks, validates the input, calculates the grade, and displays an encouraging message.

---

## 2. Project Overview

The Student Grade Calculator is a console-based Python application. It allows users to enter student details and marks, validates the entered marks, determines the grade based on predefined grading rules, and displays the result with motivational feedback.

---

## 3. Objectives

- Learn Python programming fundamentals.
- Understand if-elif-else statements.
- Learn while loops.
- Practice creating functions.
- Validate user input.
- Display results in a user-friendly format.
- Improve logical thinking and programming skills.

---

## 4. Problem Statement

Teachers and students often need a simple program to calculate grades based on marks. The objective of this project is to automate the grading process using Python.

---

## 5. Theory Concepts Used

### If-Else Statements

Used to make decisions based on conditions.

Example

```
if marks >= 90:  
    grade = "A"
```

---

### Comparison Operators

#### Operator Meaning

|    |                       |
|----|-----------------------|
| == | Equal                 |
| != | Not Equal             |
| >  | Greater Than          |
| <  | Less Than             |
| >= | Greater Than or Equal |
| <= | Less Than or Equal    |

---

### While Loop

Used for input validation.

```
while True:
```

---

### Functions

Functions help reuse code.

```
def calculate_grade(marks):
```

---

### Input Validation

Ensures marks are between 0 and 100.

---

## 6. Software Requirements

- Python 3.x
  - Visual Studio Code
  - Windows/Linux/macOS
  - Command Prompt or Terminal
- 

## 7. Hardware Requirements

- Intel i3 or above
  - 4 GB RAM
  - 100 MB Free Disk Space
- 

## 8. Project Structure

Student\_Grade\_Calculator/

```
|
|├── README.md
|├── grade_calculator.py
|├── test_cases.txt
|├── screenshots/
|   ├── Output1.png
|   ├── Output2.png
|   └── Output3.png
└── documentation.docx
```

---

## 9. Algorithm

### Step 1

Start Program

### Step 2

Enter Student Name

### Step 3

Enter Student Marks

### Step 4

Validate Marks

If marks are less than 0 or greater than 100, ask again.

### Step 5

Calculate Grade

### Step 6

Display Result

### Step 7

End Program

---

## 10. Flow of Program

Start



Enter Name



Enter Marks



Marks Valid?



No



Yes



Enter Again



Calculate Grade



Display Grade



Display Message



End

---

## 11. Python Source Code

```
def calculate_grade(marks):
    if marks >= 90:
        return "A", "Excellent! Outstanding Performance! "
    elif marks >= 80:
        return "B", "Very Good! Keep it up! "
    elif marks >= 70:
        return "C", "Good Job! Keep Learning! "
    elif marks >= 60:
        return "D", "You Passed! Practice More! "
    else:
        return "F", "Don't Give Up! Try Again! "
```

```
name = input("Enter Student Name: ")
```

```
while True:
    try:
        marks = int(input("Enter Marks (0-100): "))

        if 0 <= marks <= 100:
            break
        else:
            print("Please enter marks between 0 and 100.")

    except ValueError:
        print("Invalid input! Please enter numbers only.")
```

```
grade, message = calculate_grade(marks)
```

```
print("\n-----")
print("STUDENT RESULT")
print("-----")
print("Student Name :", name.upper())
print("Marks      :", marks)
print("Grade       :", grade)
print("Message      :", message)
print("-----")
```

---

## 12. Explanation of Code

### Function

The function `calculate_grade()` determines the student's grade based on marks.

### **While Loop**

Repeats until valid marks are entered.

### **Try-Except**

Handles invalid input such as letters or symbols.

### **If-Elif-Else**

Checks grading conditions.

### **Print Statements**

Displays the final result.

---

## **13. Sample Input & Output**

### **Example 1**

Enter Student Name: Priya

Enter Marks: 95

Output

Grade: A

Excellent! Outstanding Performance!

---

### **Example 2**

Enter Student Name: Rahul

Enter Marks: 82

Output

Grade: B

Very Good! Keep it up!

---

### **Example 3**

Enter Student Name: Ram

Enter Marks: 45

Output

Grade: F

Don't Give Up! Try Again!

---

#### 14. Test Cases

| Student | Marks | Expected Grade | Result |
|---------|-------|----------------|--------|
|---------|-------|----------------|--------|

|       |     |         |      |
|-------|-----|---------|------|
| Priya | 95  | A       | Pass |
| Rahul | 84  | B       | Pass |
| Kiran | 75  | C       | Pass |
| Sneha | 65  | D       | Pass |
| Ravi  | 40  | F       | Pass |
| Anil  | -10 | Invalid | Pass |
| Sai   | 120 | Invalid | Pass |
| Kumar | abc | Invalid | Pass |

---

#### 15. Screenshots

Include screenshots of:

- VS Code
- Python Code
- Terminal Output
- Invalid Input
- Final Result

Save inside:

screenshots/

---

#### 16. Learning Outcomes



After completing this project, I learned:

- Python syntax
  - Variables
  - User input
  - Decision making
  - If-Else statements
  - Loops
  - Functions
  - Exception handling
  - Input validation
  - Debugging programs
  - Writing reusable code
- 

## **17. Advantages**

- Easy to use.
  - Beginner-friendly.
  - Fast grade calculation.
  - Prevents invalid input.
  - Encourages students with positive messages.
  - Simple console application.
- 

## **18. Limitations**

- Console-based only.
  - Stores no student records.
  - Calculates only one student at a time.
  - Uses fixed grading rules.
- 

## **19. Future Enhancements**

- Add multiple student records.
  - Store data in a file.
  - Add percentage calculation.
  - Create a graphical user interface (GUI).
  - Connect with a database.
  - Generate report cards.
  - Export results to Excel or PDF.
- 

## **20. Conclusion**

The Student Grade Calculator is a simple Python application developed to understand basic programming concepts such as variables, user input, conditional statements, loops, functions, and exception handling. It accurately calculates student grades based on marks entered by the user and displays motivational messages according to the achieved grade. The project also validates user input to prevent invalid entries, making it more reliable and user-friendly. Overall, this project helped strengthen fundamental Python programming skills and provided practical experience in building a real-world console-based application.